

Diff-DiT: Temporal Differential Accelerator for Low-bit Diffusion Transformers on FPGA

Shidi Tang¹, Pengwei Zheng¹, Ruiqi Chen², Yuxuan Lv¹, Bruno da Silva², Ming Ling^{1,*}

¹School of Integrated Circuit, Southeast University, ²ETRO, Vrije Universiteit Brussel

*Corresponding author: trio@seu.edu.cn

Abstract—Diffusion Transformer (DiT) models have shown superior generative capabilities in image and video synthesis, yet their high computational cost during inference remains a critical bottleneck. Temporal differential computation offers a promising solution to low-bit quantization by exploiting the temporal similarity in activations. However, applying this technique to DiT's Attention layers introduces substantial memory and computation overheads.

In this paper, we present Diff-DiT, the first FPGA accelerator designed for low-bit DiT inference with differential computation. To overcome the unique challenges of DiT quantization and hardware acceleration, we propose: (1) an approximated differential attention (ADA) method that selectively approximates attention computations across time steps using a significance score, enabling low-bit on-chip execution while minimizing memory overhead; (2) an optimal cross-cast data accessing pattern with flexible data reuse to maximize computational intensity during matrix multiplications; and (3) a half-condition splitting (HCS) dataflow optimization and fine-grained pipelining to reduce the computation and memory access latency.

Extensive experiments show that Diff-DiT outperforms NVIDIA V100 GPU by 1.39× in end-to-end throughput and 5.60× in energy efficiency. When compared with state-of-the-art diffusion model accelerators, Diff-DiT also achieves 2.81× and 2.77× improvements in throughput and energy efficiency, respectively. Code is available on GitHub¹.

I. INTRODUCTION

Diffusion Transformer (DiT) models, as introduced in [1], have achieved remarkable performance within the rapidly evolving domain of Artificial Intelligence Generated Content (AIGC). These models have proven particularly effective in a range of generative visual tasks, including image synthesis [2], super-resolution [3], inpainting [4], and video generation [5]. In contrast to traditional UNet-based diffusion architectures [6], DiT models are architecturally composed of a sequence of stacked transformer blocks, which enables DiT models to benefit from the inherent scalability of transformer architectures [7], thereby enhancing their adaptability and performance across diverse tasks [8]. Nevertheless, a critical limitation of DiT models lies in their high computational complexity during inference. This challenge stems from the iterative denoising procedure, which begins with random noise and denoise it through hundreds of discretized time steps as illustrated in Figure 3.

To mitigate the computational burden, quantization techniques [9]–[11] have been proposed, which reduce the numer-

This work is supported in part by the National Natural Science Foundation of China under Grants 92464301. This research work is also supported by the Big Data Computing Center of Southeast University.

¹<https://github.com/Glnttsd/Diff-DiT>

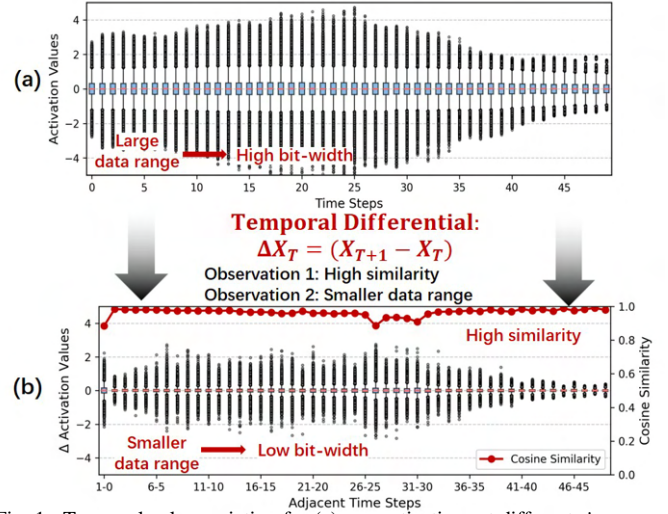


Fig. 1. Temporal value variation for (a) raw activations at different time steps (b) Δ activations between adjacent time steps.

ical precision of model parameters and activations. Conventional quantization strategies for DiT [12], [13] typically apply linear transformations to floating-point values, converting them into lower-precision integer formats such as INT8 activations with INT4 weights (A8W4). While these approaches have been effective at the algorithmic level, they often overlook the implications for hardware efficiency and practical deployment.

Emerging research on temporal differential computation [14], [15] presents a promising direction for enabling low-bit activation quantization, potentially down to 3 bits. This approach leverages the temporal redundancy inherent in diffusion models, wherein activations at adjacent time steps exhibit minimal variation [11], [12], [15], [16]. By computing the difference between current and previous activations, the resulting values demonstrate a reduced dynamic range, thereby facilitating more aggressive quantization (Figure 1 (b)) compared to the original activations (Figure 1 (a)). Despite its advantages, the differential computation paradigm is more suited to linear operations (e.g., Convolution and Linear layers), where the computed differences (Δ values) can be efficiently propagated through the network, minimizing redundant memory accesses (Figure 4 (b)). However, this method is less applicable to Attention layers [17] (Figure 4 (c)), resulting in increased memory overhead. Consequently, current differential computation-based hardware accelerators primarily target linear operations, while Attention layers remain in high bit-width, incurring significant memory transfer costs.

Field Programmable Gate Arrays (FPGAs) represent a promising approach for accelerating low-bit deep learning

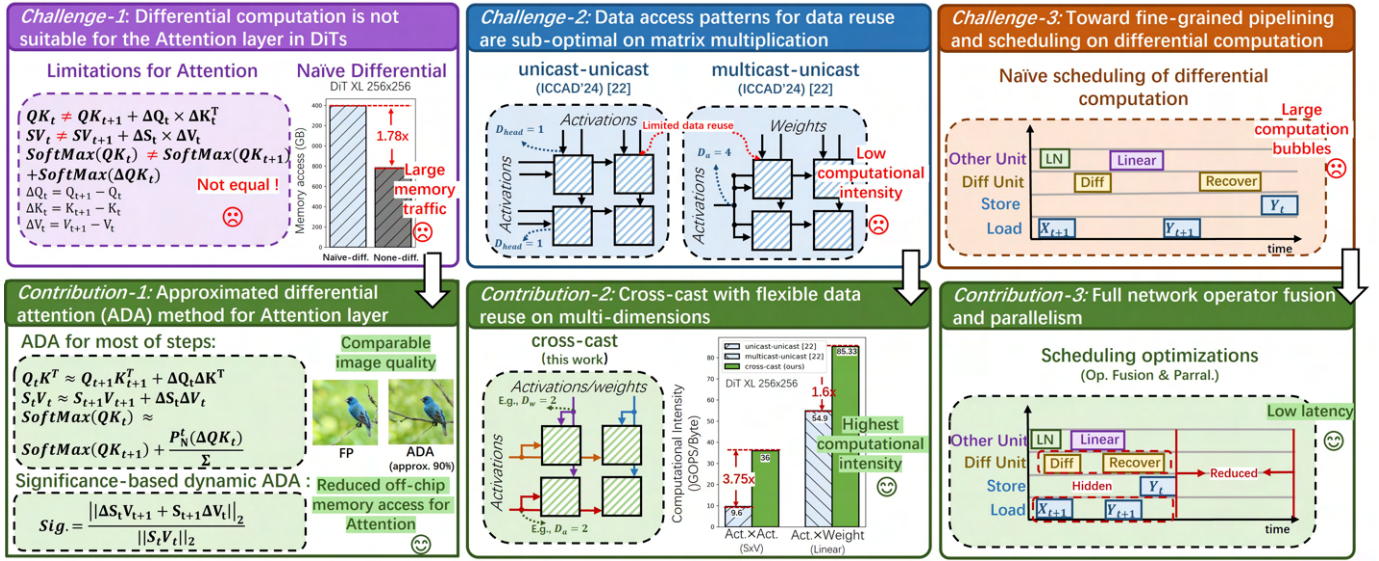


Fig. 2. Overview of the challenges and contributions.

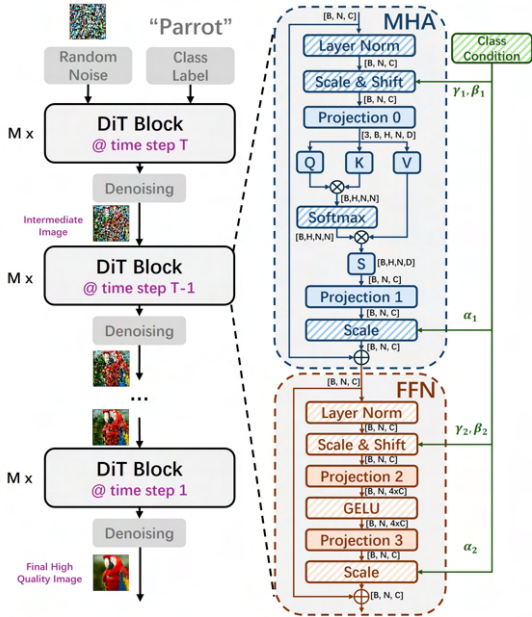


Fig. 3. Model architecture for Diffusion transformer (DiT). DiT consists of DiT blocks to denoise from the random input noise iteratively with T time steps.

models [16], [18]–[21], particularly for models quantized below 4-bit where Graphics Processing Units (GPUs) exhibit suboptimal efficiency. The reconfigurable nature of FPGAs facilitates fine-grained optimizations in execution dataflow and memory management, which proves particularly advantageous for DiT models characterized by complex computational dependencies. However, existing FPGA implementations targeting diffusion models [16], [21] demonstrate suboptimal throughput and energy efficiency, resulting in limited performance gains compared to modern GPU architectures.

To fully exploit the computational potential of FPGAs for low-bit DiT acceleration, three fundamental challenges must be addressed:

Challenge-1: The attention layer’s incompatibility with differential computation impedes both low-bit quantization and efficient memory access. As illustrated in Fig. 2 (top left), mathematical analysis reveals that the multiplication of Δ values at the current *time step* cannot be combined with the results from previous *time steps* to produce correct outputs. Naive implementation of differential computation introduces a $1.78\times$ memory transfer overhead by recovering Δ values through retrieval of previous *time step* data.

Challenge-2: Suboptimal data access patterns in matrix multiplication limit computational intensity (CI). Fig. 2 (top middle) demonstrates that previous multicast-unicast and unicast-unicast approaches [22] for *Activations* $\times$ *Weights* and *Activations* $\times$ *Activations* operations fail to fully exploit inherent data reuse, consequently increasing bandwidth requirements and reducing overall CI.

Challenge-3: Differential computation introduces scheduling complexities in computation and memory access. Fig. 2 (top right) illustrates that naive scheduling of differential operations (e.g., Diff and Recover) and memory accesses (e.g., loading X_{t+1} and Y_{t+1}) results in significant computational stalls.

To address these challenges, we propose **Diff-DiT**, an FPGA accelerator optimized for efficient inference of low-bit DiT models through differential computation. Our contributions include:

- For *Challenge-1*, we introduce an Approximated Differential Attention (ADA) method that exploits computational redundancy and temporal similarity. ADA dynamically approximates attention operations in 50%–90% of *time steps* identified as less critical to model accuracy through significance scoring, enabling on-chip differential computation while reducing memory transfer overhead by $1.82\text{--}8.12\times$.
- For *Challenge-2*, we propose an analytical model to optimize data parallelism in matrix multiplications, sub-

sequently proposing a cross-cast data access pattern with flexible data reuse. This approach yields $1.55\text{-}5.41\times$ and $3.63\text{-}5.98\times$ improvements in computational intensity compared to conventional multicast-unicast and unicast-unicast patterns, respectively.

- For *Challenge-3*, we implement comprehensive dataflow optimization through pipelining and parallelism. Our inter-module Half-Condition Splitting (HCS) method and intra-module scheduling optimizations for the differential operations further reduce end-to-end latency by $1.16\text{-}1.21\times$.
- Extensive evaluation demonstrates that Diff-DiT achieves the average of $1.39\times/5.60\times$ and $2.77\times/2.81\times$ improvements in throughput/energy efficiency comparing to NVIDIA V100 GPU and state-of-the-art diffusion model accelerators.

II. PRELIMINARIES AND RELATED WORKS

A. Diffusion Transformers

Figure 3 shows the architecture of the Diffusion Transformer (DiT) [1], which is built from a sequence of DiT blocks. Each block mainly consists of a multi-head attention (MHA) module and a feed-forward network (FFN) module. The model takes Gaussian random noise as input, along with a class label indicating the target image category. The input is first normalized using LayerNorm and then scaled and shifted by parameters γ and β from the class conditioning layer. After that, a Linear layer maps the input to multi-head query (Q), key (K), and value (V) vectors. The attention is computed as $\text{Attention}(Q, K, V) = \max \text{Max} \left(\frac{QK^T}{\sqrt{D}} \right) V$, where D is a scaling constant and is omitted for simplicity in the rest of this work. And $\text{SoftMax}(x^i) = \frac{\exp(x^i)}{\sum_j \exp(x^j)}$ for each element i along the last dimension of x . We denote the result of SoftMax as S . The attention output is then projected and scaled by a factor α before being passed to the FFN, which includes two Linear layers, a LayerNorm, and a GELU activation. Most of the projection and attention operations (except SoftMax) can be efficiently implemented with matrix-matrix multiplications. After passing through M DiT blocks, the output is sent to a denoising module. Over T time steps, the model gradually reconstructs a high-quality image. In this work, we mainly focus on the computationally expensive DiT blocks.

B. Differential Computation and Its Challenge

Differential computation reduces redundant operations by computing the difference between two similar values. The difference, denoted as Δ , is then quantized as $\Delta_q = \text{round}(\Delta/s + z)$, where s and z represent the scale factor and zero point, respectively. Since quantization error decreases when the data range is smaller [23], Δ can be quantized with a lower bit-width due to its typically narrower range (as shown in Fig. 1). This facilitates efficient low-precision computation. Figure 4 illustrates how differential computation operates along the temporal dimension. In Fig. 4(a), given the input activation

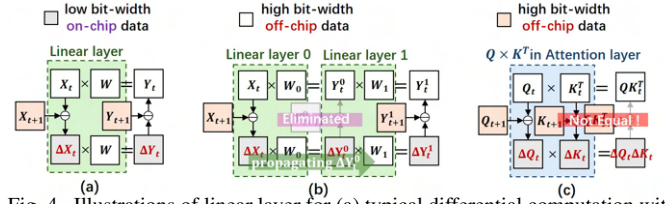


Fig. 4. Illustrations of linear layer for (a) typical differential computation with low-bit calculation (b) the propagation property of differential computation that eliminates the intermediate memory access transfer. And (c) typical differential computation can not be directly applied to Attention layer.

X_t of a Linear layer at time step t , where $Y_t = X_t \times W$, the difference between X_{t+1} and X_t is computed as ΔX_t . Multiplying ΔX_t with W yields ΔY_t . The original output Y_{t+1} can then be recovered by adding ΔY_t to the previous output Y_t . However, this method incurs additional memory access overhead, as both X_{t+1} and Y_{t+1} must be fetched from off-chip DRAM into the on-chip buffer [15].

The propagated differential computation approach alleviates the memory overhead when multiple differential computations are sequentially cascaded. As shown in Fig. 4 (b), if two Linear layers (denote as f_0 and f_1) are executed consecutively, the intermediate difference ΔY_t^0 produced by f_0 can be directly passed to f_1 for further differential computation. This is based on the relation $Y_t^1 = f_1(f_0(X_t)) = f_1(f_0(X_{t+1}) + f_0(\Delta X_t)) = Y_{t+1}^1 + f_1(f_0(\Delta X_t))$. As a result, there is no need to load Y_t^0 from memory, effectively reducing memory overhead.

However, this propagation approach cannot be directly applied to the Attention layers in DiT models. As shown in Fig. 4 (c), for the computation of $Q_t \times K_t^T$, the relation $QK_t \neq QK_{t+1} + \Delta Q_t \times \Delta K_t^T$ holds, where $\Delta Q_t = Q_{t+1} - Q_t$ and $\Delta K_t = K_{t+1} - K_t$. The same issue occurs in the computation of $S_t \times V_t$. Moreover, non-linear functions like SoftMax and GELU are not compatible with differential computation, since $\text{SoftMax}(QK_t) \neq \text{SoftMax}(QK_{t+1}) + \text{SoftMax}(\Delta QK_t)$ and $\text{GELU}(X_t) \neq \text{GELU}(X_{t+1}) + \text{GELU}(\Delta X_t)$.

C. Existing Diffusion Model Accelerators

Recent studies have extensively investigated the hardware acceleration of Diffusion Models on both FPGA platforms [16], [21], [24] and ASIC architectures [14], [15], [25]. Among these efforts, approaches based on differential computation, such as Cambricon-D [15] and Ditto [14], have demonstrated significant advances in low-bit quantization and computational acceleration. Nevertheless, both Cambricon-D and Ditto are limited in their applicability to Attention layers, failing to support differentiation within either UNet-based structures or transformer-based architectures such as DiT models. Other works like FlightVGM [16] and EXION [25] investigate sparsity within diffusion models across both temporal and spatial dimensions. Notably, these sparsification techniques are complementary to the proposed differential computation approach, and their integration has the potential to further enhance the overall performance of differential accelerators.

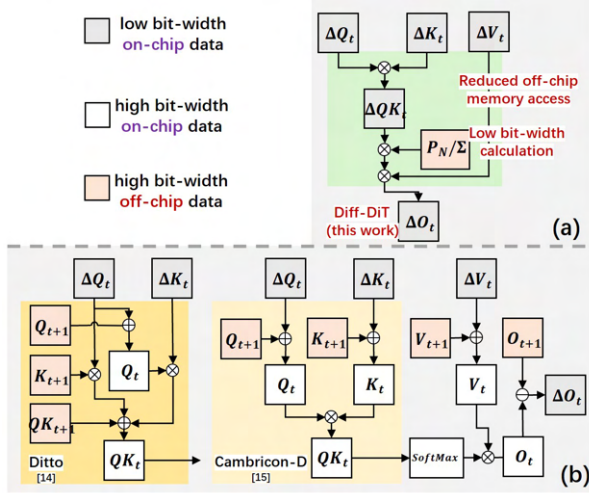


Fig. 5. Differential dataflow of Attention layer for (a) Diff-DiT with approximated differential attention (ADA) method and (b) Ditto [14] and Cambricon-D [15].

III. APPROXIMATED DIFFERENTIAL ATTENTION

To address the limitations outlined in Section II-B, we introduce an approximated differential attention (ADA) method, which enables the approximate computation of Attention layers under differential computation for a substantial proportion (**50%–90%**) of total *time steps*. The design of ADA is inspired by two key observations, as illustrated in Fig. 1. **First**, the activations between adjacent *time steps* exhibit a high degree of similarity, with cosine similarity exceeding 0.9 in most cases. **Second**, the Δ values possess a significantly smaller magnitude compared to the original activations. Consequently, approximating these small Δ terms has a negligible impact on image quality, as illustrated in Fig. 2 (lower left). Furthermore, recent studies have highlighted considerable redundancy within diffusion models [26], where techniques such as cache reuse or block skipping [27], [28] have been successfully employed to reduce computational cost without compromising model accuracy.

In particular, for the computation of $Q \times K^T$ and $S \times V$, ADA approximates by omitting the calculation of the two terms $\Delta Q_t \times K_{t+1}^T + Q_t \times \Delta K_t^T$ and $\Delta S_t \times V_{t+1} + S_t \times \Delta V_t$. This leads to the following formulation:

$$\Delta QK_t = QK_{t+1} - QK_t \approx \Delta Q_t \times \Delta K_t^T \quad (1)$$

$$\Delta SV_t = SV_{t+1} - SV_t \approx \Delta S_t \times \Delta V_t \quad (2)$$

which satisfies the propagation property inherent to differential computation. To further enhance the applicability of differential computation within Attention layers, we propose a differentiable SoftMax operation, termed Diff-SFM, which exploits the data similarity between adjacent *time steps*. Specifically, Diff-SFM approximates the denominator of the *SoftMax* function by leveraging the observation that $\sum_j e^{QK_t^j} \approx \sum_j e^{QK_{t+1}^j}$ due to the minor variation of QK values across consecutive *time steps*. Additionally, the exponential function in the numerator, $f(x) = e^x$, is approximated using a piecewise linear function $\phi(x) = p_n x + c_n$, where $n \in (0, 1, \dots, N)$

denotes the index of each piece, and N represents the total number of segments used for approximation. As a result, the *SoftMax* output at *time step* t can be approximated as:

$$\Delta S_t \approx \frac{\phi(QK_{t+1}^i) - \phi(QK_t^i)}{\sum_j e^{QK_{t+1}^j}} \approx \frac{P_N^{t+1}}{\sum_j e^{QK_{t+1}^j}} \odot \Delta QK_t \quad (3)$$

where $\Delta S_t = \text{SoftMax}(QK_{t+1}) - \text{SoftMax}(QK_t)$, P_N^{t+1} is the matrix of slopes p_n in function $\phi(x)$ at the *time step* of $t + 1$. $\odot$ denotes the element-wise matrix multiplication. Notably, P_N^{t+1} and $\sum_j e^{QK_{t+1}^j}$ are constants since they were calculated at previous *time step* of $t + 1$, we keep them in fixed 16-bit precision and load them from off-chip memory.

Comparing with Cambricon-D [15] and Ditto [14], our ADA method enables fully propagated differential computation for the Attention layer because the subtracted results of Attention layer between two adjacent *time steps* can be calculated as:

$$\begin{aligned} \Delta \text{Attention}(Q_t, K_t, V_t) &= SV_{t+1} - SV_t \\ &\approx \Delta S_t \times \Delta V_t \quad (\text{using Eq. 2}) \\ &\approx \left(\frac{P_N^{t+1}}{\sum_j e^{QK_{t+1}^j}} \odot \Delta QK_t \right) \times \Delta V_t \quad (\text{using Eq. 3}) \\ &\approx \left(\frac{P_N^{t+1}}{\sum_j e^{QK_{t+1}^j}} \odot (\Delta Q_t \times \Delta K_t^T) \right) \times \Delta V_t \quad (\text{using Eq. 1}) \end{aligned} \quad (4)$$

where all multiplications between Δ matrices can be performed in **low bit-width precision with minimal memory transfer overhead** for loading the constants. We illustrate and compare the differential dataflow of Attention layer for Diff-DiT with existing differential accelerators in Fig. 5, where both Cambricon-D and Ditto require extensive memory loading from off-chip to on-chip to recover the Δ values into raw values.

Nevertheless, directly applying these approximations to all *time steps* within the DiT model may lead to a substantial degradation in the quality of the generated images, primarily due to the accumulation of approximation errors across both model blocks and temporal dimensions. To alleviate this issue, we propose a selective approximation strategy, wherein approximation is applied only to *time steps* exhibiting lower approximation error, as determined by a significance criterion:

$$\text{Significance} = \frac{\|\Delta S_t \times V_{t+1} + S_{t+1} \Delta V_t\|_2}{\|S_t \times V_t\|_2} \quad (5)$$

This criterion is computed offline and quantifies the relative impact of the approximated terms compared to the original computation without approximation. A lower *Significance* value indicates reduced approximation error, suggesting that ADA can be safely applied. In addition, we extend the differential computation paradigm to the GELU activation function by introducing an approximate differential GELU (Diff-GELU), where the non-linear activation is approximated using a piecewise linear function $w(x) = g_n x + b_n$. The

corresponding slopes across different pieces are represented as a matrix G_N . Consequently, the differential output of the GELU function at *time step* t can be approximated as $\Delta GELU_t = G_N^{t+1} \times \Delta X_t$. This formulation enables the propagation of differential computation within the FFN module. Experimental results in Section VI-B show that ADA can be effectively applied to the majority of *time steps* **without incurring noticeable degradation in image quality**.

IV. HARDWARE ARCHITECTURE

A. System Overview

As illustrated in Fig. 6, the system architecture of **Diff-DiT** consists of off-chip HBM, a memory controller, an on-chip instruction scheduler, a BRAM-based global buffer, and multiple specialized computation units. The computation units include: the Class Condition Unit, the Normal SFU (Special Function Unit) (e.g., GELU, SoftMax (SFM), LayerNorm (LN), and Residual connection (Res)), the Differential SFU (e.g., Diff-SFM and Diff-GELU), the Quantization/Dequantization Unit, the Diff/Recover unit and the Systolic Array (SA). The Diff-SFM unit loads $P_N^{t+1} / \sum_j e^{QK_{i+1}^j}$ stored in step $t+1$, and calculates ΔS_t according to Eq. (3). Similarly, the Diff-GELU unit loads G_N^{t+1} stored in step $t+1$ and multiplies it with ΔX_t to calculate $\Delta GELU_t$. The Diff/Recover unit is responsible for generating (or recovering from) Δ values by computing the difference between two raw values and subsequently quantizing the result. The systolic array contains multiple 16×16 SA tiles and supports unified matrix multiplication across four distinct cases (i.e., $8b \times 8b$, $8b \times 4b$, $3b \times 4b$, $3b \times 3b$) by leveraging DSP packing techniques [29]. We optimize the data access pattern of matrix multiplications across multiple SA tiles, as discussed in Section IV-D.

The system operates under two distinct dataflows, selected dynamically at different *time steps* to balance model accuracy and hardware efficiency, as discussed in Section III. Specifically, the *Differential Dataflow* is employed for layers processed using differential computation and our ADA method, where Linear layers compute with INT3 Δ activations and INT4 weights (denoted as A3W4), while Attention layers (e.g., $Q \times K^T$ and $S \times V$) compute with A3A3. The *Normal Dataflow* is used for non-differential computations, where Linear layers compute with A8W4, while Attention layers compute with A8A8. These two dataflows are collectively managed within a *Hybrid Dataflow*. The instruction scheduler orchestrates the switching between dataflows at runtime.

B. Differential Dataflow

All Linear layers in DiT models are executed using the standard differential dataflow, as depicted in Fig. 4 (a), where the Δ values are propagated directly to the subsequent layers. The Attention layers adopt our proposed ADA method illustrated in Fig. 5 (a) to enable approximate differential computation. Unlike previous methods, our ADA framework enables fully propagated differential computation across the entire DiT model, eliminating the need to recover Δ values into high-bit-width raw values with minimal off-chip data transfer overhead.

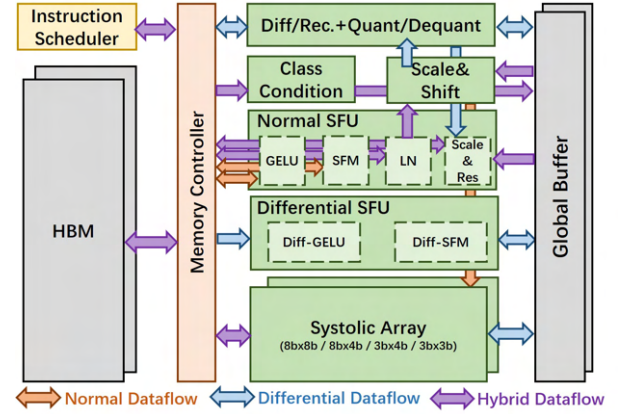


Fig. 6. System architecture of Diff-DiT

For other non-computational intensive operations, including LayerNorm, Scale, Shift, and Class Conditioning layers, we retain a fixed 16-bit precision to preserve model accuracy. In these cases, the Δ values are first recovered to the original precision values before computation.

C. Design Space Exploration

To efficiently perform large matrix multiplications for the Linear and Attention layers, it is critical to design optimal data access and sharing strategies across multiple SA tiles. **A suboptimal data access pattern can significantly reduce computational intensity (GOPS/Byte)** due to low data reuse, which increases memory demands, and inefficient data parallelism, which leads to padding and wasteful computations, ultimately diminishing performance.

For a detailed exploration of the data access pattern, we consider matrix multiplication $A \times W$, with matrix sizes of $[S_{batch}, S_a^{row}, S_a^{col}]$ for matrix A and $[S_{batch}, S_w^{row}, S_w^{col}]$ for matrix W . We then construct an analytical model designed to maximize computational intensity (CI), given by:

$$\begin{aligned} \underset{D_{batch}, D_a, D_w}{\operatorname{argmax}} \quad CI &= \frac{T \times U}{M_a + M_w} \\ \text{s.t.} \quad N_{total}^{dsp} &\leq N_{max}^{dsp} \end{aligned} \quad (6)$$

Here, D_{batch} , D_a , and D_w represent the degrees of data reuse across SA tiles for the batch (or head), the rows of matrix A , and the columns of matrix W , respectively. For instance, as shown in Fig. 2 (bottom middle), the data reuse values $D_a = 2$ and $D_w = 2$ are chosen so that activations A and weights W are reused across two SA tiles, respectively. The total number of DSPs employed, N_{total}^{dsp} , is calculated as follows:

$$N_{total}^{dsp} = D_{batch} \times D_a \times D_w \times N_{SA}^{dsp} \quad (7)$$

where N_{SA}^{dsp} represents the number of DSPs allocated to each SA tile, and N_{max}^{dsp} is the total number of DSPs available on the FPGA board. The computation throughput, T (measured in GOPS), is given by:

$$T = N_{total}^{dsp} \times N_{dsp_pack} \times f \quad (8)$$

where f denotes the operating frequency of the SA tiles, and N_{dsp_pack} represents the DSP packing factor. The utilization

TABLE I
DEGREES OF REUSE WITH THE HIGHEST COMPUTATIONAL INTENSITY
WITH CROSS-CAST PATTERN FOR DiT [1] XL 256×256 MODEL

Dataflow Type	Computation Type	D_{batch}	D_a	D_w
Normal Dataflow	Linear (A8W4)	1	8	4
	QxK (A8A8)	1	8	4
	SxV (A8A8)	1	8	4
Differential Dataflow	Linear (A3W4)	1	4	8
	QxK (A3A3)	1	4	8
	SxV (A3A3)	2	2	8

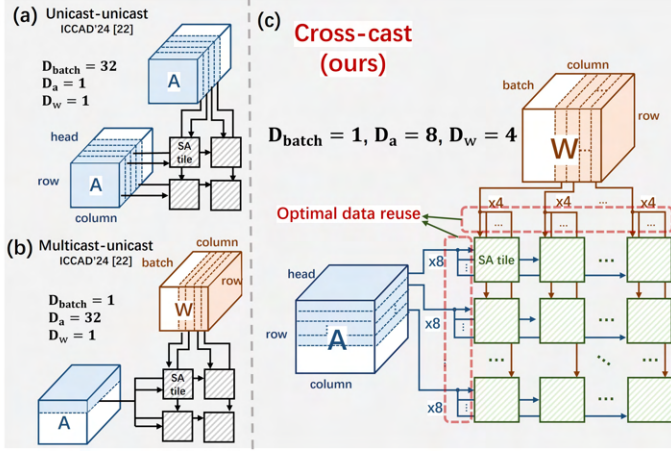


Fig. 7. Data access patterns for (a) unicast-unicast, (b) multicast-unicast, and (c) our cross-cast with flexible data reuse on multi-dimensions.

rate U represents the percentage of effective computation, accounting for the impact of data padding, and is computed as:

$$U = \frac{S_a^{row} \times S_a^{col} \times S_w^{col} \times D_{batch}}{N_{total}^{dsp} \times P_a \times P_w \times R_a \times R_w \times S_a^{col}} \times 100\% \quad (9)$$

where R_a and R_w denote the number of repeated data accesses for matrices A and W , respectively, due to tiled matrix multiplication. These are calculated by:

$$R_a = \left\lceil \frac{S_w^{col}}{D_a \times S_{SA}^{col} \times P_w} \right\rceil, R_w = \left\lceil \frac{S_a^{row}}{D_w \times S_{SA}^{row} \times P_a} \right\rceil \quad (10)$$

Here, P_a and P_w represent the number of data packed for matrices A and W , respectively, while S_{SA}^{row} and S_{SA}^{col} denote the size of a single SA tile. The memory bandwidth requirements for matrices A and W , denoted by M_a and M_w , are calculated as:

$$M_a = D_{batch} \times D_w \times S_{SA}^{row} \times P_a \times B_a / 8 \times f \quad (11)$$

$$M_w = D_{batch} \times D_a \times S_{SA}^{col} \times P_w \times B_w / 8 \times f$$

where B_a and B_w denote the bit-widths of matrices A and W , respectively.

D. Cross-cast Pattern

By solving the optimization problem discussed in Section IV-C, we introduce the *cross-cast* data access pattern for matrix multiplication, which maximizes computational intensity. As illustrated in Fig. 7 (c), the cross-cast pattern provides a flexible degree of data reuse across multiple dimensions,

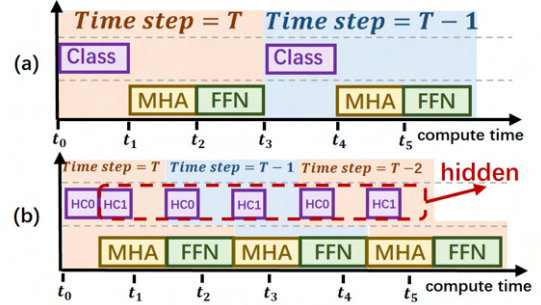


Fig. 8. Inter-module (a) naive scheduling and (b) optimized scheduling by our Half-Condition Split (HCS) method. The Class Condition module is split into HC0 and HC1 modules.

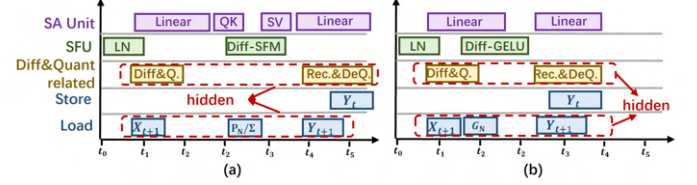


Fig. 9. Intra-module fine-grained pipelining of differential computation related operations for the (a) MHA and (b) FFN modules.

specifically $\{D_{batch}, D_a, D_w\}$. For example, in case of the DiT XL 256×256 model, we determine $\{D_{batch} = 1, D_a = 8, D_w = 4\}$ for both *Activation*×*Weights* and *Activation*×*Activation* under the *Normal Dataflow*. This configuration ensures that each row (column) of matrix A (matrix W) is shared across 8 (4) SA tiles.

In the case of the *Differential Dataflow*, we derive alternative solutions and present the optimal configurations with the highest computational intensity in Table I. Furthermore, we compare our flexible cross-cast pattern with other fixed patterns, such as unicast-unicast [22] and multicast-unicast [22], which are characterized by limited data reuse for weights or activations (i.e., $D_a = 1$ or $D_w = 1$). Quantitative results from these comparisons are provided in Section VI-C.

V. SCHEDULING

We fully optimize the dataflow of Diff-DiT by leveraging pipelining and parallelism. Following the approach outlined in [22], [30], we decompose the DiT block into two key modules: the Multi-Head Attention (MHA) and the Feed-Forward Network (FFN) module, as illustrated in Fig. 3. We first introduce the *Half-Condition Splitting* (HCS) method to hide the inter-module latency associated with the Class Condition computation, as shown in Fig. 9 (a). The HCS method divides the original computation into two half-conditions, HC0 and HC1, where HC0 (HC1) computes $\alpha_1(\alpha_2)$, $\beta_1(\beta_2)$, and $\gamma_1(\gamma_2)$, which are then used by the subsequent MHA (FFN) module. At the first *time step* t , the MHA computation begins before the completion of HC0, operating in parallel with HC1, which will later be used in the FFN module. As the FFN module performs its computation, HC0 for the next *time step* $t - 1$ is computed concurrently, and this parallelism continues for subsequent *time steps*.

To mitigate the overhead introduced by differential computation, we further optimize the intra-module dataflow for both

the MHA and FFN modules, as shown in Fig. 9 (a) and (b). By pipelining the differentiation and quantization (Diff&Q.), recovery and dequantization (Rec.&DeQ.), along with additional memory accesses (X_{t+1} , P_N/Σ , G_N , Y_{t+1}), we significantly improve computational efficiency. For simplicity, we do not provide the details for loading weights or storing intermediate activations in Fig. 9.

VI. EVALUATION

A. Experimental Setup

Models and Datasets. We evaluate Diff-DiT on the widely studied DiT [1] XL/2 pretrained model for the image generation task under different total *time steps* (50, 100, and 250) on the ImageNet 256×256 and 512×512 datasets [31]. Note that Diff-DiT can be applied to higher resolutions since Diff-DiT follows an overlay design approach [32].

Baselines. We compare Diff-DiT with the following baselines: ❶ **NVIDIA V100 GPU** Pytorch implementation with FP16 precision. We measure the power consumption with nvidia-smi. Notably, we set the batch size to 1 under all implementations as in other works [33], [34] for a fair comparison. ❷ **Cambricon-D** [15] and **Ditto** [14], both of which are differential computation-based ASIC designs. We re-implement these methods by differentiating and quantizing the activations in INT3 (INT4) precision for Cambricon-D (Ditto), based on the FP16 precision (A8W8 precision [10]) version. For a fair comparison with ASIC designs, we follow previous works [16], [20], scaling the hardware configurations (peak computing performance in GOPS, memory bandwidth, and on-chip memory resources) of ASIC designs to match those of Diff-DiT. Power consumption is also scaled accordingly. ❸ **FlightVGM** [16], a state-of-the-art FPGA design for diffusion transformer based video generation models. FlightVGM uses a sparsification method that is orthogonal to our quantization method, and we scale its performance using the same number of DSPs as our design based on the performance reported in their paper. We only report the relative performance improvement against the same GPU baseline since FlightVGM does not support DiT model. Notably, we do not compare the model accuracy with FlightVGM because of the difference of tasks and optimization technique.

Algorithm Configuration. We randomly generate 5k images from the model and evaluate image quality using commonly used metrics Fréchet Inception Distance (FID), spatial FID (sFID), and Inception Score (IS), using the ADM toolkit [35]. We build upon the state-of-the-art quantization method PTQ4DiT [12] and further extend the quantization into the differential computation dataflow. We use uniform channel-wise and tensor-wise quantization for all activations and weights, respectively. For the linear approximation of the exponential function in SoftMax and GELU function, we set the number of pieces $N = 8$, based on our experiments. The selected *time steps* based on the significance criterion is obtained by the evaluation on 100 classes (1k images) from the datasets.

Hardware Configuration. Diff-DiT is implemented on the AMD V80 FPGA using high-level synthesis (HLS) with

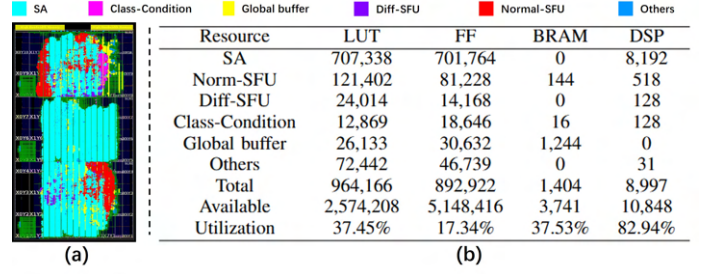


Fig. 10. (a) Implementation layout and (b) resource utilization on AMD V80 FPGA.

TABLE II
MODEL ACCURACY COMPARISONS OF DiT [1] XL ON IMAGENET 256×256 WITH DIFFERENTIAL-BASED QUANTIZATION METHODS

Method	Time steps	Bit-width		FID↓	sFID↓	IS↑
		Linear (A/W)	Attention (A/A)			
FP	50	16/16		24.58	40.26	179.83
Cambricon-D [15]		3/16	16/16	29.11	45.87	120.5
Ditto [14]		4/8	4/8	58.17	49.51	105.07
Ours (50% approx)		3/4(50%) + 8/4(50%)	3/3(50%) + 8/8(50%)	28.03	43.07	129.44
FP		16/16		24.92	40.41	143.22
Cambricon-D [15]	100	3/16	16/16	26.9	41.59	128.62
Ditto [14]		4/8	4/8	38.04	39.86	99.05
Ours (67% approx)		3/4(67%) + 8/4(33%)	3/3(67%) + 8/8(33%)	26.77	41.31	130.59
FP		16/16		24.53	37.19	201.31
Cambricon-D [15]		3/16	16/16	25.98	38.21	161.01
Ditto [14]	250	4/8	4/8	28.68	36.31	157.77
Ours (90% approx)		3/4(90%) + 8/4(10%)	3/3(90%) + 8/8(10%)	23.15	38.88	151.17

250 MHz. Power consumption is obtained using the Xilinx Power Estimator (XPE) tool, and hardware resource utilization (shown in Fig. 10) is obtained after synthesis and implementation in Vivado 2024.1. We evaluate hardware performance in terms of speedups and energy efficiency when comparing with other hardware designs. Additionally, we adapt the DSP packing technique [29] that packs 2 multiplications for A8A8 or A8W4, and 6 multiplications for A3W4 or A3A3 on one DSP58 resource.

B. Algorithm Evaluation

Guided by the significance criterion, we exhaustively explore the percentage of *time steps* where the ADA method is applied. The optimal results are summarized in Table II. The image quality is comparable to the FP16 baseline when 50%, 67%, 90% of the total 50, 100, 250 *time steps* are computed under *Differential Dataflow* with our ADA method. Notably, as the total *time steps* increase, the percentage of approximation increases, which aligns with the characteristics of diffusion models: larger models with more *time steps* exhibit higher computational redundancy [26], [28]. We also compare our method with other differential computation based approaches, Cambricon-D and Ditto, in Table II. Our method achieves comparable or better results for most metrics, even with lower bit-width. Additionally, we obtained similar results under 512×512 resolution, although these results are omitted due to page limitations.

C. Hardware Evaluation

Speedup and Energy Efficiency. As illustrated in Figure 11, Diff-DiT achieves the average of 1.39× performance and 5.60× energy efficiency improvement when comparing with high performance NVIDIA V100 GPU. This is due to the low-bit (especially down to 3-bit) computation, the fine-grained control of dataflow, and pipelining that current GPU

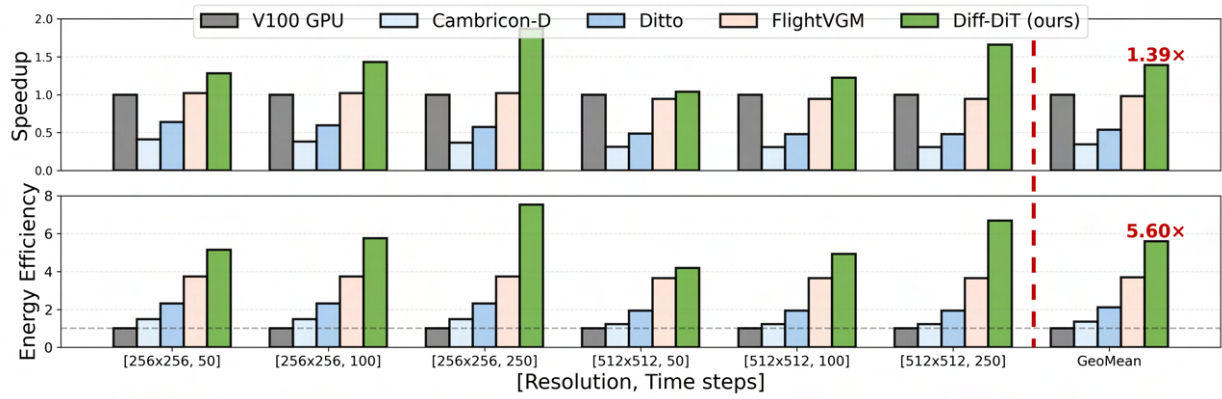


Fig. 11. Speedup and energy efficiency comparisons between NVIDIA V100 GPU, differential computation based accelerators Cambricon-D [15], Ditto [14], and SOTA FPGA accelerator FlightVGM [16]. Speedup and energy efficiency are normalized to the GPU baseline.

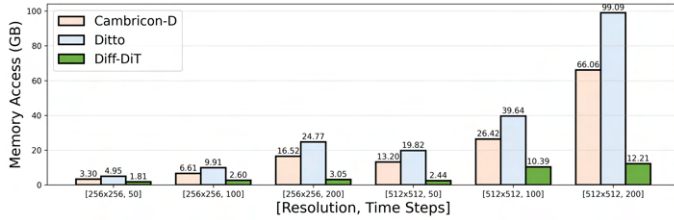


Fig. 12. Memory access comparisons of the Attention later between Diff-DiT and two differential computation-based designs: Cambricon-D and Ditto.



Fig. 13. Design points of different data access pattern, dataflow, and computation type for DiT model with (a) 256x256 and (b) 512x512 resolutions. Diff-DiT achieves the highest computational intensity with our cross-cast pattern.

architecture can not efficiently handle. Diff-DiT also achieves $4.17\times/2.70\times$ and $4.20\times/2.69\times$ improvement on the performance and energy efficiency comparing with Cambricon-D/Ditto. Moreover, Diff-DiT achieves $1.44\times$ and $1.55\times$ improvement on performance and energy efficiency compared with FlightVGM. This is because Diff-DiT achieves lower bit efficient computation and largely reduces the memory access overhead introduced by differential computation.

Memory Access. We compare the memory access overhead of Cambricon-D, Ditto, and Diff-DiT for the differential computation of Attention layer in Figure 12. Our approximated differential attention method reduces the memory loading from off-chip to recover the Δ values into raw values, resulting in $1.82\text{-}5.42\times$ reduction comparing to Cambricon-D and $2.73\text{-}8.12\times$ reduction comparing to Ditto across various time steps and resolutions.

Computational Intensity. Figure 13 illustrates the design choices of different data access patterns when performing

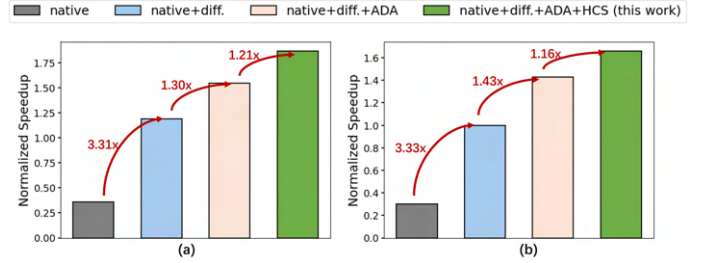


Fig. 14. Ablation studies for the differential computation (diff.), the approximated differential attention (ADA), and HCS scheduling optimizations under the resolutions of (a) 256x256 and (b) 512x512 resolutions. The speedups are normalized to the GPU baseline.

matrix multiplications (i.e., Linear, $Q \times K^T$, $S \times V$) on the systolic array. Our cross-cast achieves the highest computational intensity among other access patterns under both *Normal Dataflow* and *Differential Dataflow*. We achieve $1.55\text{--}5.41\times$ and $3.63\text{--}5.98\times$ improvement compared with the multicast-unicast [22] and unicast-unicast [22] access patterns.

D. Ablation Study

As illustrated in Fig. 14, the native hardware design (without any optimizations) is about 66% slower than the GPU baseline. For DiT under 250 steps, the differential computation method brings $3.31\times$ and $3.33\times$ speedups under the resolution of 256x256 and 512x512, due to the low-bit efficient computation. Our ADA method additionally brings $1.30\times$ and $1.43\times$ speedups by approximating and reducing the memory transfer overhead. Further, our HCS method and scheduling optimizations brings $1.21\times$ and $1.16\times$ speedups by exploring pipelining and parallelism.

VII. CONCLUSION

We propose Diff-DiT, an efficient FPGA accelerator for DiT models with temporal differential computing. First, we propose an approximated differential attention to mitigate the previous challenge of differentiating the Attention layer. Second, we propose a cross-cast data access pattern that achieves the highest computational intensity when performing matrix multiplications. Third, we optimize the dataflow by exploring the parallelism (with our HCS method) and pipelining. Diff-DiT achieves $1.39\times$ speedup and $5.60\times$ energy efficiency improvement when compared to the NVIDIA V100 GPU.

REFERENCES

- [1] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 4195–4205.
- [2] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, and K. Aberman, "Dreambooth: Fine tuning text-to-image diffusion models for subject-driven generation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 22 500–22 510.
- [3] J. Ho, C. Saharia, W. Chan, D. J. Fleet, M. Norouzi, and T. Salimans, "Cascaded diffusion models for high fidelity image generation," *Journal of Machine Learning Research*, vol. 23, no. 47, pp. 1–33, 2022.
- [4] A. Lugmayr, M. Danelljan, A. Romero, F. Yu, R. Timofte, and L. Van Gool, "Repaint: Inpainting using denoising diffusion probabilistic models," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 11 461–11 471.
- [5] W. Harvey, S. Naderiparizi, V. Masrani, C. Weilbach, and F. Wood, "Flexible diffusion modeling of long videos," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 953–27 965, 2022.
- [6] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 10 684–10 695.
- [7] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *International conference on machine learning*. PMLR, 2021, pp. 10 347–10 357.
- [8] S. Gao, P. Zhou, M.-M. Cheng, and S. Yan, "Masked diffusion transformer is a strong image synthesizer," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 23 164–23 173.
- [9] T. Zhao, T. Fang, E. Liu, R. Wan, W. Soedarmadji, S. Li, Z. Lin, G. Dai, S. Yan, H. Yang *et al.*, "Vidit-q: Efficient and accurate quantization of diffusion transformers for image and video generation," *arXiv preprint arXiv:2406.02540*, 2024.
- [10] X. Li, Y. Liu, L. Lian, H. Yang, Z. Dong, D. Kang, S. Zhang, and K. Keutzer, "Q-diffusion: Quantizing diffusion models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 17 535–17 545.
- [11] Y. Shang, Z. Yuan, B. Xie, B. Wu, and Y. Yan, "Post-training quantization on diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 1972–1981.
- [12] J. Wu, H. Wang, Y. Shang, M. Shah, and Y. Yan, "Ptq4dit: Post-training quantization for diffusion transformers," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [13] L. Chen, Y. Meng, C. Tang, X. Ma, J. Jiang, X. Wang, Z. Wang, and W. Zhu, "Q-dit: Accurate post-training quantization for diffusion transformers," *arXiv preprint arXiv:2406.17343*, 2024.
- [14] S. Kim, H. Lee, W. Cho, M. Park, and W. W. Ro, "Ditto: Accelerating diffusion model via temporal value similarity," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 338–352.
- [15] W. Kong, Y. Hao, Q. Guo, Y. Zhao, X. Song, X. Li, M. Zou, Z. Du, R. Zhang, C. Liu *et al.*, "Cambricon-d: Full-network differential acceleration for diffusion models," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 903–914.
- [16] J. Liu, S. Zeng, L. Ding, W. Soedarmadji, H. Zhou, Z. Wang, J. Li, J. Li, Y. Dai, K. Wen *et al.*, "Flightvgn: Efficient video generation model inference with online sparsification and hybrid precision on fpgas," in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 2–13.
- [17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [18] Y. Bai, H. Zhou, K. Zhao, H. Wang, J. Chen, J. Yu, and K. Wang, "Fet-opu: A flexible and efficient fpga-based overlay processor for transformer networks," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2023, pp. 1–9.
- [19] M. Sun, Z. Li, A. Lu, Y. Li, S.-E. Chang, X. Ma, X. Lin, and Z. Fang, "Film-qnn: Efficient fpga acceleration of deep neural networks with intra-layer, mixed-precision quantization," in *Proceedings of the 2022 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, 2022, pp. 134–145.
- [20] S. Zeng, J. Liu, G. Dai, X. Yang, T. Fu, H. Wang, W. Ma, H. Sun, S. Li, Z. Huang *et al.*, "Flightllm: Efficient large language model inference with a complete mapping flow on fpgas," in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2024, pp. 223–234.
- [21] G. Yang, Y. Xie, Z. J. Xue, S.-E. Chang, Y. Li, P. Dong, J. Lei, W. Xie, Y. Wang, X. Lin *et al.*, "Sda: Low-bit stable diffusion acceleration on edge fpgas," in *2024 34th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2024, pp. 264–273.
- [22] Y. Ji, C. Fang, S. Ma, H. Shao, and Z. Wang, "Co-designing binarized transformer and hardware accelerator for efficient end-to-end edge deployment," in *2024 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 2024.
- [23] R. Banner, Y. Nahshan, and D. Soudry, "Post training 4-bit quantization of convolutional networks for rapid-deployment," *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [24] H. Zhou, Y. Liu, H. Wang, E. Tang, S. Li, Y. Zhang, G. Dai, Y. Liu, and K. Wang, "Deploying diffusion models with scheduling space search and memory overflow prevention based on graph optimization," in *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, 2025, pp. 170–176.
- [25] J. Heo, A. Putra, J. Yoon, S. Yune, H. Lee, J.-H. Kim, and J.-Y. Kim, "Exion: Exploiting inter-and intra-iteration output sparsity for diffusion models," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 324–337.
- [26] K. Zheng, C. Lu, J. Chen, and J. Zhu, "Dpm-solver-v3: Improved diffusion ode solver with empirical model statistics," *Advances in Neural Information Processing Systems*, vol. 36, pp. 55 502–55 542, 2023.
- [27] X. Ma, G. Fang, and X. Wang, "Deepcache: Accelerating diffusion models for free," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 15 762–15 772.
- [28] S. Li, T. Hu, J. van de Weijer, F. Shahbaz Khan, T. Liu, L. Li, S. Yang, Y. Wang, M.-M. Cheng *et al.*, "Faster diffusion: Rethinking the role of the encoder for diffusion model inference," *Advances in Neural Information Processing Systems*, vol. 37, pp. 85 203–85 240, 2024.
- [29] J. Sommer, M. A. Özkan, O. Keszocze, and J. Teich, "Dsp-packing: Squeezing low-precision arithmetic into fpga dsp blocks," in *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2022, pp. 160–166.
- [30] T. Wang, L. Gong, C. Wang, Y. Yang, Y. Gao, X. Zhou, and H. Chen, "Via: A novel vision-transformer accelerator based on fpga," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 11, pp. 4088–4099, 2022.
- [31] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009, pp. 248–255.
- [32] H. Chen, J. Zhang, Y. Du, S. Xiang, Z. Yue, N. Zhang, Y. Cai, and Z. Zhang, "Understanding the potential of fpga-based spatial acceleration for large language model inference," *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, no. 1, pp. 1–29, 2024.
- [33] R. Sarkar, S. Abi-Karam, Y. He, L. Sathidevi, and C. Hao, "Flowggn: A dataflow architecture for real-time workload-agnostic graph neural network inference," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 1099–1112.
- [34] B. Zhang, R. Kannan, and V. Prasanna, "Boostgcn: A framework for optimizing gcn inference on fpga," in *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2021, pp. 29–39.
- [35] P. Dhariwal and A. Nichol, "Diffusion models beat gans on image synthesis," *Advances in neural information processing systems*, vol. 34, pp. 8780–8794, 2021.